

Итоговая Аттестация

Сардаана Огонерова

Группа: КИСП-23

В первую очередь был прописан класс SQLHelper необходимый для взаимодействия с локальной БД.

```
public class SQLHelper extends SQLiteOpenHelper { 8 usages
    public SQLHelper (Context context){ 4 usages
        super(context, name: "PassM6RDB", factory: null, version: 1);
    }
    @Override
    public void onCreate(SQLiteDatabase db){
        db.execSQL("create table " +
            "Users (id integer primary key autoincrement, " +
            "Login text, Password text);");
        db.execSQL("create table " +
            "PaswordRecords (id integer primary key autoincrement, UserId integer, " +
            "Service text, Login text, Password text, Note text);");
    }
    @Override 10 usages
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion){
        db.execSQL("drop table if exists Users");
        db.execSQL("drop table if exists PaswordRecords");
        onCreate(db);
    }
    private Cursor getUsersTable(){ 1 usage
        SQLiteDatabase db = this.getWritableDatabase();
        return db.rawQuery( sql: "select * from Users", selectionArgs: null);
    }
    public User getUser(String login){ 2 usages
        SQLiteDatabase db = this.getWritableDatabase();
        Cursor cursor = db.rawQuery( sql: "select * from Users where Login = '"+login+"'", selectionArgs: null);
        User user = new User();
        user.setId(-1);
        if(cursor != null){
            while (cursor.moveToNext()){
                user = new User(cursor.getInt( columnIndex: 0), cursor.getString( columnIndex: 1),
                    cursor.getString( columnIndex: 2));
            }
        }
        return user;
    }
    private Cursor getPasswordRecordsTable(int userId){ 1 usage
        SQLiteDatabase db = this.getWritableDatabase();
        return db.rawQuery( sql: "select * from PaswordRecords where UserId = '"+userId, selectionArgs: null);
    }
}
```

Рисунок1

```

public void insertUser(String login,String pass){ 1 usage
    SQLiteDatabase db = this.getWritableDatabase();
    db.execSQL("insert into Users (Login, Password) values ('+ login +', '"+pass+"')");
}

public void insertPasswordRecord(int userId,String service,String login,String pass, String note){ 1 usage
    SQLiteDatabase db = this.getWritableDatabase();
    db.execSQL("insert into PaswordRecords (UserId,Service,Login," +
        " Password, Note) values ('+userId+', '"+ service +', '"+login+"', '"+pass+"', '"+note+"')");
}

public ArrayList<User> getUsersList(){ no usages
    ArrayList<User> users = new ArrayList<>();
    Cursor cursor = getUsersTable();
    if(cursor != null){
        while (cursor.moveToNext()){
            users.add(new User(cursor.getInt( columnIndex: 0),cursor.getString( columnIndex: 1),
                cursor.getString( columnIndex: 2)));
        }
    }
    return users;
}

public ArrayList<PasswordRecord> getPasswordRecordsList(int userId){ 3 usages
    ArrayList<PasswordRecord> passwordRecords = new ArrayList<>();
    Cursor cursor = getPasswordRecordsTable(userId);
    if(cursor != null){
        while (cursor.moveToNext()){
            passwordRecords.add(new PasswordRecord(cursor.getInt( columnIndex: 0),
                cursor.getInt( columnIndex: 1), cursor.getString( columnIndex: 2),
                cursor.getString( columnIndex: 3),
                cursor.getString( columnIndex: 4), cursor.getString( columnIndex: 5)));
        }
    }
    return passwordRecords;
}

```

Рисунок 2

В данном классе определены 2 таблицы: Users и PaswordRecords

Первая таблица имеет поля Id, Login, Pasword. И предназначена для реализации авторизации, регистрации.

Вторая таблица имеет поля: Id, UserId, Service, Login, Password, Note.

Данная таблица хранит записи о паролях конкретных пользователей

Для обеих таблиц определено по 3 метода:

getИмяТаблицыTable – возвращает объект cursor.

insertИмяТаблицы – вставка данных.

getИмяТаблицыList – возвращает список записей полученных из cursor

Так же для обеих таблиц были определены модели:

User и PasswordRecord

```

package com.example.attestations;

public class User { 9 usages
    private int Id; 3 usages
    private String Login; 3 usages
    private String Password; 3 usages
    public User(){} 1 usage

    public User(int Id,String Login, String Password){ 2 us
        this.Id = Id;
        this.Login = Login;
        this.Password = Password;
    }

    public int getId() {
        return Id;
    }

    public String getLogin() { no usages
        return Login;
    }

    public String getPassword() { 1 usage
        return Password;
    }

    public void setId(int id) {
        Id = id;
    }

    public void setLogin(String login) { no usages
        Login = login;
    }

    public void setPassword(String password) { no usages
        Password = password;
    }
}

```

Рисунок 3

```

package com.example.attestations;

public class PasswordRecord { 9 usages
    private int Id; 3 usages
    private int UserId; 3 usages
    private String Service; 3 usages
    private String Login; 3 usages
    private String Password; 3 usages
    private String Note; 3 usages
    public PasswordRecord(){} no usages
    public PasswordRecord(int Id,int UserId, String Service, String Login, String Password, String Note){...}
    public int getId() {
        return Id;
    }
    public String getLogin() { return Login; }
    public String getPassword() { return Password; }
    public void setId(int id) { Id = id; }
    public void setLogin(String login) { Login = login; }
    public void setPassword(String password) { Password = password; }
    public String getService() { 1 usage
        return Service;
    }
    public void setService(String service) { no usages
        Service = service;
    }
    public int getUserId() { no usages
        return UserId;
    }
    public void setUserId(int userId) { no usages
        UserId = userId;
    }
    public String getNote() { 3 usages
        return Note;
    }
    public void setNote(String note) { no usages
        Note = note;
    }
}

```

Рисунок 4

Каждая из 2х моделей обладает полями соответствующими столбцам таблицы, двумя конструкторами (пустым и на все параметры), геттерами и сеттерами.

Далее была прописана активность авторизации

The image shows a simple login form with a light purple background. It contains two text input fields. The first field is labeled 'Login' and the second is labeled 'Passord'. Below these fields are two rounded rectangular buttons. The left button is labeled 'Войти' (Login) and the right button is labeled 'Регистрация' (Registration).

Рисунок5

```

public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        ETPassword = findViewById(R.id.editTextPassword);
        BLogin = findViewById(R.id.button);
        BRegistration = findViewById(R.id.button2);
        sqlHelper = new SQLHelper(getApplicationContext());
        BLogin.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String login = ETLogin.getText().toString();
                String pass = ETPassword.getText().toString();
                if(login.isEmpty()){
                    Toast.makeText(getApplicationContext(), text: "Введите логин", Toast.LENGTH_SHORT).show();
                    return;
                }
                if(pass.isEmpty()){
                    Toast.makeText(getApplicationContext(), text: "Введите пароль", Toast.LENGTH_SHORT).show();
                    return;
                }
                User user = sqlHelper.getUser(login);
                if(user != null){
                    if(pass.equals(user.getPassword())){
                        Intent intent = new Intent(getApplicationContext(), PasswordManagerActivity.class);
                        intent.putExtra(name: "UserId", user.getId());
                        startActivity(intent);
                    }
                    else {
                        Toast.makeText(getApplicationContext(), text: "Не верный пароль", Toast.LENGTH_SHORT).show();
                    }
                }
                else{
                    Toast.makeText(getApplicationContext(), text: "Не верный логин", Toast.LENGTH_SHORT).show();
                }
            }
        });
        BRegistration.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(getApplicationContext(), RegistrationActivity.class);
                startActivity(intent);
            }
        });
    }
}

```

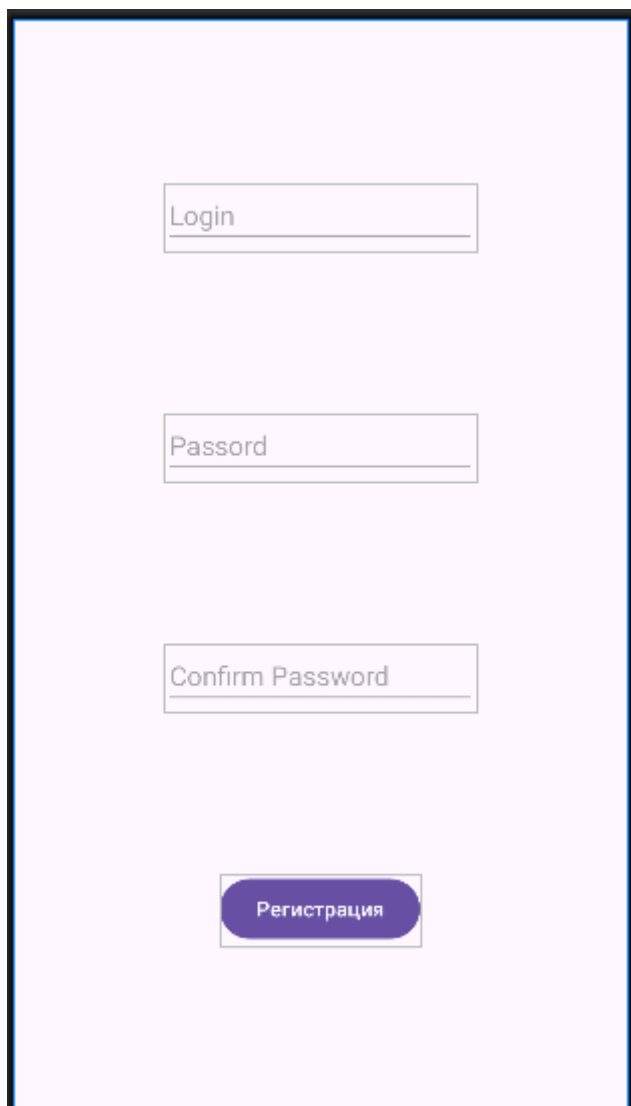
Рисунок 6

На данной активности производится авторизация перед которой производится проверка на пустые поля в случае если поля пустые, не верного логина или пароля выводятся соответствующие сообщения.

Сверка логинов и паролей и логинов производится с данными хранящимися в переменной user которая получает данные из бд.

При успешной авторизации производится переход на активность PasswordManagerActivity, а так же в intent вставляется Extra("UserId",user.getId()) для передачи Id пользователя.

Также с данной формы имеется возможность перехода на форму регистрации.



The image shows a registration form with a light purple background and a dark blue border. It contains three text input fields stacked vertically, each with a placeholder label: 'Login', 'Passord', and 'Confirm Password'. Below these fields is a dark blue button with rounded corners and the text 'Регистрация' in white.

Рисунок 7

```

public class RegistrationActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        ETLogin = findViewById(R.id.editTextLog);
        ETPassword = findViewById(R.id.editTextPass);
        ETConfirmPassword = findViewById(R.id.editTextConfirmPass);
        BRegistration = findViewById(R.id.button3);
        sqlHelper = new SQLHelper(getApplicationContext());
        BRegistration.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String login = ETLogin.getText().toString();
                String pass = ETPassword.getText().toString();
                String confPass = ETConfirmPassword.getText().toString();
                if(login.isEmpty()){
                    Toast.makeText(getApplicationContext(), text: "Введите логин", Toast.LENGTH_SHORT).show();
                    return;
                }
                if(pass.isEmpty()){
                    Toast.makeText(getApplicationContext(), text: "Введите пароль", Toast.LENGTH_SHORT).show();
                    return;
                }
                if(confPass.isEmpty()){
                    Toast.makeText(getApplicationContext(), text: "Введите подтверждение пароля", Toast.LENGTH_SHORT).show();
                    return;
                }
                if(!confPass.equals(pass)){
                    Toast.makeText(getApplicationContext(), text: "Пароли не совпадают", Toast.LENGTH_SHORT).show();
                    return;
                }
                User user = sqlHelper.getUser(login);
                if(user.getId() == -1){
                    Log.w( tag: "REEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE", msg: "dfsdfdf");
                    sqlHelper.insertUser(login, pass);
                    Intent intent = new Intent(getApplicationContext(), MainActivity.class);
                    startActivity(intent);
                }
                else{
                    Toast.makeText(getApplicationContext(), text: "Логин занят", Toast.LENGTH_SHORT).show();
                }
            }
        });
    }
}

```

Рисунок 8

В коде данной активности перед регистрацией пользователя производится проверка на пустые поля, а также на совпадение текста поля пароля и текста поля подтверждения пароля. Далее с помощью запроса к бд проверяется свобода логина, если логин свободен то данные записываются в бд и производится переход на форму авторизации.

Далее была прописана PasswordManagerActivity

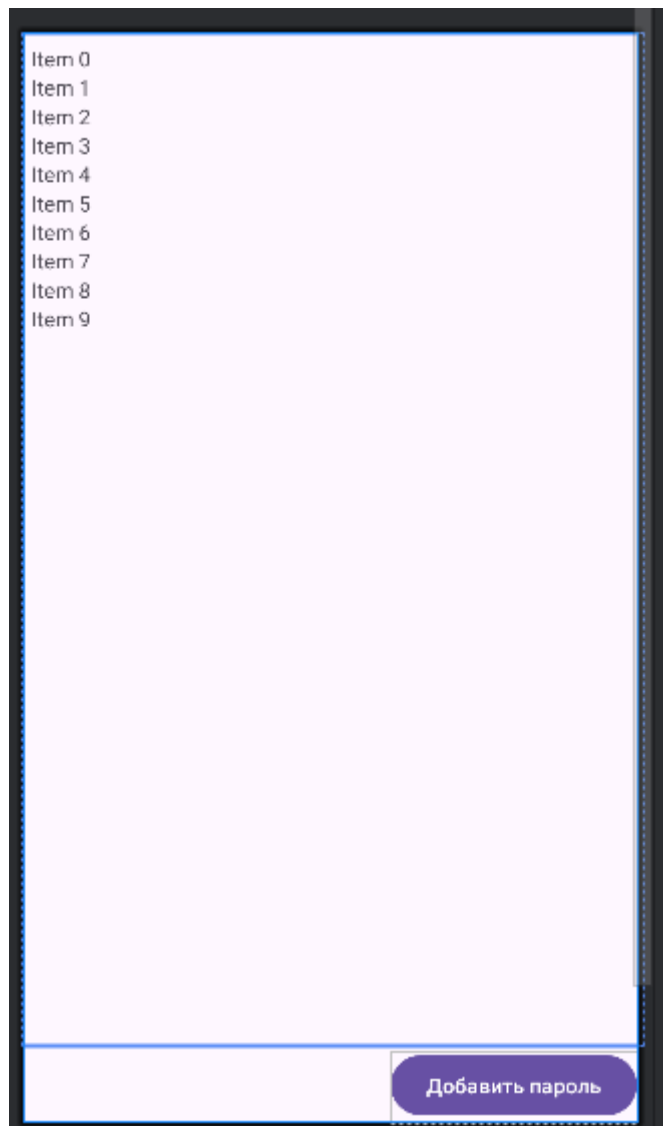


Рисунок 9


```

public class PasswordManagerActivity extends AppCompatActivity {
    private RecyclerView recyclerView; 5 usages
    private TextView emptyMessage; 3 usages
    private Button addButton; 2 usages
    private PasswordRecordAdapter adapter; 2 usages
    private SQLHelper dbHelper; 2 usages
    private int currentUserId; // Предположим, пользователь авторизован 4 usages
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_password_manager);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        currentUserId = getIntent().getIntExtra( name: "UserId", defaultValue: 0);
        Log.w( tag: "PasswordManagerActivity", msg: "UserId: "+currentUserId);
        recyclerView = findViewById(R.id.recycleView);
        emptyMessage = findViewById(R.id.emptyText);
        addButton = findViewById(R.id.button4);
        dbHelper = new SQLHelper( context: this);
        ArrayList<PasswordRecord> records = dbHelper.getPasswordRecordsList(currentUserId);

        if (records.isEmpty()) {
            emptyMessage.setVisibility(View.VISIBLE);
            recyclerView.setVisibility(View.GONE);
        } else {
            emptyMessage.setVisibility(View.GONE);
            recyclerView.setVisibility(View.VISIBLE);
            adapter = new PasswordRecordAdapter(records);
            recyclerView.setLayoutManager(new LinearLayoutManager( context: this));
            recyclerView.setAdapter(adapter);
        }

        addButton.setOnClickListener(v -> {
            Intent intent = new Intent( packageContext: this, AddPassActivity.class);
            intent.putExtra( name: "UserId", currentUserId);
            startActivity(intent);
        });
    }
}

```

Рисунок 10

Для отображения записей паролей используется Recycler View, для отображения сообщения о том, что записи не добавлены используется textView в зависимости от того есть записи у данного пользователя или их нет видимым становится либо Recycler View , либо textView. Также на форме расположена кнопка для перехода в Activity добавления записи пароля. Как и случае перехода от активности авторизации к активности менеджера паролей в эктра записывается Id вошедшего пользователя.

Для отображения записей в recycler View были созданы item_password_record.xml являющийся макетом записи и PasswordRecordAdapter.java отвечающий логику работы и построения записей.

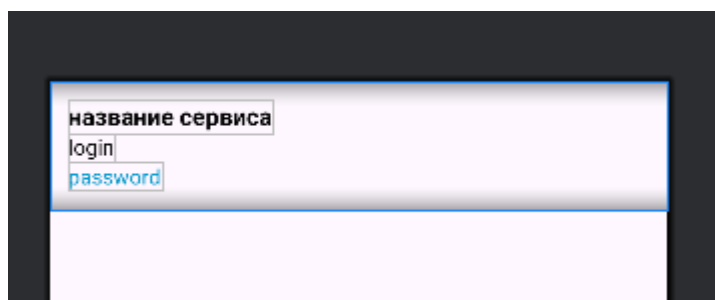


Рисунок 11

```

public class PasswordRecordAdapter extends RecyclerView.Adapter<PasswordRecordAdapte

private ArrayList<PasswordRecord> records; 3 usages
private Set<Integer> expandedItems = new HashSet<>(); 3 usages

public PasswordRecordAdapter(ArrayList<PasswordRecord> records) { 1 usage
    this.records = records;
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    View view = LayoutInflater.from(parent.getContext())
        .inflate(R.layout.item_password_record, parent, attachToRoot: false);
    return new ViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
    PasswordRecord record = records.get(position);

    holder.serviceText.setText(record.getService());
    holder.loginText.setText(record.getLogin());

    boolean expanded = expandedItems.contains(position);
    holder.passwordText.setText(expanded ? record.getPassword() : ".....");

    if (expanded && record.getNote() != null && !record.getNote().isEmpty()) {
        holder.noteText.setText(record.getNote());
        holder.noteText.setVisibility(View.VISIBLE);
    } else {
        holder.noteText.setVisibility(View.GONE);
    }

    holder.itemView.setOnClickListener(v -> {
        if (expanded) {
            expandedItems.remove(position);
        } else {
            expandedItems.add(position);
        }
        notifyItemChanged(position);
    });
}
}

```

Рисунок 12

```

@Override
public int getItemCount() {
    return records.size();
}

public static class ViewHolder extends RecyclerView.ViewHolder { 4 usages
    TextView serviceText, loginText, passwordText, noteText; 2 usages

    public ViewHolder(@NonNull View itemView) { 1 usage
        super(itemView);
        serviceText = itemView.findViewById(R.id.serviceText);
        loginText = itemView.findViewById(R.id.loginText);
        passwordText = itemView.findViewById(R.id.passwordText);
        noteText = itemView.findViewById(R.id.noteText);
    }
}
}

```

Рисунок 13

Item служит для определения элементов интерфейса и их отображения, адаптер отвечает за наполнение элементов интерфейса item-а данными и функционалом.

Так в методе onBindViewHolder производится присвоение данных полям, а также назначения слушателя нажатия на сам item по нажатию пароль становится виден пароль и заметка если она не пуста. Данные берутся из List<PasswordRecord> переданного в адаптер из PasswordManagerActivity в которой он был получен из бд с помощью соответствующего метода.

Далее была создана AddPassActivity

Сервис или сайт

Логин

Пароль

Заметка (необязательно)

Сохранить

Рисунок 13

```

public class AddPassActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        serviceEditText = findViewById(R.id.edit_service);
        loginEditText = findViewById(R.id.edit_login);
        passwordEditText = findViewById(R.id.edit_password);
        noteEditText = findViewById(R.id.edit_note);
        togglePasswordVisibility = findViewById(R.id.toggle_password_visibility);
        saveButton = findViewById(R.id.btn_save);
        dbHelper = new SQLHelper(context: this);
        userId = getIntent().getIntExtra(name: "UserId", defaultValue: 0);
        togglePasswordVisibility.setOnClickListener(v -> {
            if (isPasswordVisible) {
                passwordEditText.setInputType(InputType.TYPE_CLASS_TEXT | InputType.TYPE_TEXT_VARIATION_PASSWORD);
                togglePasswordVisibility.setImageResource(R.drawable.baseline_visibility_off_24);
            } else {
                passwordEditText.setInputType(InputType.TYPE_CLASS_TEXT | InputType.TYPE_TEXT_VARIATION_VISIBLE_PASSWORD);
                togglePasswordVisibility.setImageResource(R.drawable.baseline_visibility_24);
            }
            passwordEditText.setSelection(passwordEditText.getText().length());
            isPasswordVisible = !isPasswordVisible;
        });
        saveButton.setOnClickListener(v -> savePasswordRecord());
    }

    private void savePasswordRecord() { 1 usage
        String service = serviceEditText.getText().toString().trim();
        String login = loginEditText.getText().toString().trim();
        String password = passwordEditText.getText().toString().trim();
        String note = noteEditText.getText().toString().trim();

        if (service.isEmpty() || login.isEmpty() || password.isEmpty()) {
            Toast.makeText(context: this, text: "Заполните обязательные поля", Toast.LENGTH_SHORT).show();
            return;
        }

        List<PasswordRecord> prevCount = dbHelper.getPasswordRecordsList(userId);
        dbHelper.insertPasswordRecord(userId, service, login, password, note);
        List<PasswordRecord> nowCount = dbHelper.getPasswordRecordsList(userId);
        if (nowCount.size() > prevCount.size()) {
            Toast.makeText(context: this, text: "Пароль сохранён", Toast.LENGTH_SHORT).show();
            Intent intent = new Intent(packageContext: this, PasswordManagerActivity.class);
            intent.putExtra(name: "UserId", userId);
            startActivity(intent);
        } else {
            Toast.makeText(context: this, text: "Ошибка при сохранении", Toast.LENGTH_SHORT).show();
        }
    }
}

```

Рисунок 14

На данной активности прописана проверка обязательных полей на пустоту перед добавлением записи.

Если обязательные поля заполнены то происходит добавление записи в бд и переход на активность менеджера паролей в txttrs всё также вставляется Id пользователя. Так же предусмотрен функционал просмотра скрытого пароля при нажатии на image View с картинкой перечёркнутого или открытого глаза. Добавление так сопровождается сообщениями.